EMBKFS

Embedded Modern Block File System

On-Disk Format Specification

Version 2.0

Foundations & Core Structures

A from-scratch, copy-on-write, checksummed filesystem designed for the EmbLink operating system.

Contents

1 Introduction

EMBKFS (Embedded Modern Block File System) is a 64-bit, copy-on-write filesystem built from scratch for the EmbLink operating system. It is the OS's native filesystem. It is deliberately not intended for the EmbLink real-time kernel, whose constrained targets are better served by a small, purpose-built embedded filesystem.

This document specifies the **on-disk format** — the exact arrangement of bytes that defines an EMBKFS volume — together with the reasoning behind each decision. The on-disk format is the true specification; any implementation that reads and writes these bytes correctly is a conforming implementation.

The design lineage is ZFS and Btrfs: all metadata lives in copy-on-write B-trees, integrity is protected by a Merkle tree of checksums, and crash-consistency follows from atomic pointer replacement rather than journalling of data. Where this document states a value as **reserved**, that field must be written as zero and an implementation must not assume any meaning for it.

> **Scope.** Version 2.0 covers the complete on-disk format: addressing model, superblock, the universal block pointer and key, both tree-node layouts, and the three core item types (inode, directory entry, extent). The runtime machinery (allocator, the copy-on-write commit path, transactions, snapshots) is the subject of later specifications and is only previewed here.

2 Design Principles

Five principles govern every decision in this specification. They are stated up front because each individual choice below is an application of one of them.

- **Design from invariants, not features.** The format is defined by a small set of structures and rules that, taken together, make advanced features possible. Features such as snapshots and compression are consequences of a correct core, not independent additions.
- **Copy-on-write is the soul.** Live data is never overwritten in place. Updates write new blocks and atomically install them by replacing a single pointer. Crash-consistency and snapshots fall out of this directly.
- **Integrity is end-to-end.** Every block is verified against a checksum held by its parent, forming a Merkle tree rooted in the superblock. File data is checksummed as well as metadata, giving detection of silent corruption.

- **Align and reserve over pack tightly.** Structures are sized for clean alignment and carry reserved space for forward growth. A few wasted bytes now are worth avoiding a format-breaking change later.
- **Evolve without breaking old volumes.** A three-class feature-flag mechanism lets the format grow for years while old implementations continue to handle newer volumes as safely as possible.

3 Addressing Model

An EMBKFS volume is divided into fixed-size **blocks**. The block size is chosen at format time; the default is 4 KiB, matching the CPU page size (which benefits future memory-mapping of files). Permitted block sizes are 4, 8, 16, 32, and 64 KiB.

Every location on disk is addressed by a 64-bit **block number**. Blocks are numbered from 0. A 64-bit block number at the 4 KiB default addresses 64 zebibytes of storage — far beyond any practical target — so block numbers never need to widen.

EMBKFS sits on top of the EmbLink block-device layer, which performs I/O in 512-byte sectors. An EMBKFS block of `block_size` bytes therefore spans `block_size / 512` device sectors. Reading EMBKFS block `N` is:

```
embk_block_read(dev, N * (block_size/512), block_size/512, buf)
```

Endianness. All multi-byte values in EMBKFS are stored **little-endian** on disk, regardless of the CPU. The format defines the byte order; an implementation on a big-endian CPU byte-swaps on read and write. This makes a volume portable across architectures by definition, which matters because EmbLink targets both x86-64 and (later) ARM64.

4 Volume Layout

The superblock is the bootstrap structure: it is the one place an implementation can read without already knowing the layout. Because of this chicken-and-egg constraint, its location must be fixed and known to the code in advance.

4.1 Superblock placement

- **Primary superblock** at a fixed byte offset of `65536` (64 KiB) from the start of the device. The offset is in bytes, not blocks, because at mount time the block size is not yet known — it is stored in the very superblock being read. A byte offset can be sought to with zero prior knowledge. The 64 KiB position also leaves room for boot code and avoids the contested first sectors used by partition tables and other filesystems.

- **Backup superblock** in the last block of the device. Its location is computed from the device's own reported size (the block-device layer knows the sector count), not from a value stored in the primary. This is deliberate: the backup must be findable precisely when the primary is unreadable. Placing it at the opposite end of the device gives it physical independence from the primary.

4.2 Mount procedure

On mount, an implementation performs the following, in order:

```
1. Read 8 bytes at byte offset 65536. If != EMBKFS_MAGIC -> not EMBKFS, abort.
2. Verify the superblock checksum. If it fails, fall back to the backup
   superblock at the last block (located via the device's reported size).
3. If both copies are valid but differ, the one with the higher generation wins.
4. unknown = feature_incompat & ~KNOWN_INCOMPAT
   if unknown != 0  -> refuse to mount.
5. unknown = feature_ro_compat & ~KNOWN_RO_COMPAT
   if unknown != 0  -> mount READ-ONLY.
6. feature_compat: unknown bits are ignored; mount read-write.
7. Backstop: if version_major > MAX_KNOWN_MAJOR -> refuse.
```

5 Superblock

The superblock identifies the volume, negotiates format compatibility, and anchors the metadata tree. Its first field is the magic number EMBKFS_MAGIC, an 8-byte value that, stored little-endian, reads **EMBKFS17** in a hex dump:

```
// Packed so little-endian storage yields bytes 45 4D 42 4B 46 53 31 37
// ('E','M','B','K','F','S','1','7'), reading "EMBKFS17" in a hex dump.
#define EMBKFS_MAGIC  0x373153464B424D45ULL
```

> **On the magic.** An 8-byte magic makes accidental collision with random data astronomically unlikely (1 in ~1.8e19). Encoding readable ASCII makes the superblock instantly recognizable when debugging. The 17 is an identifying tag for the author's project, not a version number — the version lives in its own fields below.

5.1 Version and feature negotiation

A single linear version number cannot express *what* changed between revisions, so any change becomes an all-or-nothing wall. EMBKFS instead splits compatibility into three independent feature bitmasks, classified by how dangerous it is for an implementation to not understand a given feature:

- feature_compat — unknown bits are safe to ignore; mount read-write.

- feature_ro_compat — unknown bits mean reading is safe but writing is not; mount read-only.
- feature_incompat — unknown bits mean the volume cannot be safely touched; refuse to mount.

Every feature added over the life of the format is assigned a bit in exactly one of these masks, chosen by answering: *what happens if an implementation does not understand it?* This is the mechanism that lets a version-1 volume still mount on a version-20 implementation, and lets a version-20 volume be handled as safely as possible by a version-1 implementation.

5.2 Superblock fields

All pointers in the superblock are full block pointers (Section 6), so they carry the checksum and generation of their target — the root of the Merkle tree of trust.

Offset	Size	Field	Meaning
0	8	magic	EMBKFS_MAGIC; identifies the volume.
8	4	version_major	Format generation; bumped only for sweeping redesigns.
12	4	version_minor	Informational revision; never affects mountability.
16	8	feature_compat	Compatible feature bits (unknown = ignore).
24	8	feature_ro_compat	RO-compatible feature bits (unknown = mount read-only).
32	8	feature_incompat	Incompatible feature bits (unknown = refuse).
40	8	block_size	Bytes per block (4096, 8192, ... 65536).
48	8	total_blocks	Total blocks in the volume.
56	8	free_blocks	Approximate free block count (hint; authoritative free space is in the allocator structures).
64	16	uuid	Volume UUID (identity across devices).
80	8	generation	Superblock generation; the newest valid copy has the highest value.
88	32	root	Block pointer to the root of the top-level metadata tree.
120	32	checkpoint	Block pointer to the most recent checkpoint (recovery).
152	8	checksum	Checksum of the superblock (covers all preceding bytes).

Note: field offsets above are the logical design layout; the implementation fixes the exact C struct and verifies its size. The superblock occupies the start of its block; the remainder of the block to block_size is reserved.

6 The Block Pointer

A pointer in EMBKFS is not a bare block number. It is a 32-byte **fat pointer** that carries everything needed to locate a block, verify its integrity, and reason about which copy-on-write generation produced it. Every internal tree link and the superblock's root are block pointers.

```
struct embk_block_ptr {
    uint64_t block;      // block number of the target on disk
    uint64_t checksum;   // checksum of the target's contents (CRC32C in low 32 bits, v1)
    uint64_t generation; // generation/transaction that wrote the target
    uint64_t flags;      // per-pointer flags (reserved in v1, must be 0)
};
// = 32 bytes. Exactly 128 pointers fit in a 4 KiB block.
```

6.1 Why the checksum lives in the parent

A block cannot vouch for itself: if it is corrupt, a checksum stored inside it may be corrupt too. EMBKFS therefore stores a block's checksum in the **pointer that references it** — that is, in its parent. Following a pointer reads the child and verifies it against the checksum the parent supplied. The parent's own checksum is in its parent, and so on up to the root, whose checksum is in the superblock (itself checksummed and duplicated).

The result is a Merkle tree: every block is verified by the level above it, and corruption anywhere breaks the chain visibly. This is the structural basis of end-to-end integrity.

6.2 Checksum algorithm and reserved width

Version 1 uses **CRC32C**, which is hardware-accelerated on both x86-64 and ARM and excellent at catching the accidental corruption (bad sectors, torn writes) that v1 targets. The checksum field is nonetheless a full 64 bits, with CRC32C occupying the low 32. Reserving 64 bits means a future upgrade to a 64-bit hash can be made through a feature flag **without changing the on-disk layout** — forward-compatibility by reserving space.

6.3 The generation field

The generation records which transaction wrote the target. It serves snapshots (a block older than a snapshot is shared with it and must not be freed) and integrity (a node's own recorded generation is cross-checked against the generation the pointer claims, catching a stale pointer to a block that was freed and reused). The `flags` field is reserved in v1 for future per-pointer attributes such as marking a target as compressed or encrypted.

7 The Key

Every record in the metadata tree is addressed by a 24-byte composite **key**. A single key type addresses inodes, directory entries, extents, and extended attributes alike. The key is what allows one tree to hold an entire filesystem.

```
struct embk_key {
    uint64_t object_id;   // which object this record belongs to
    uint64_t type;        // record kind (1 byte used; upper 7 reserved, must be 0)
    uint64_t offset;      // polymorphic: interpreted per 'type'
};
// = 24 bytes. Compared field-by-field: object_id, then type, then offset.
```

7.1 Comparison order and locality

Keys are compared by object_id first, then type, then offset. This ordering produces locality, and the locality is the whole point:

- **object_id first** clusters every record about one object together in the tree — its inode, its directory entries or extents, its attributes. “Everything about object N” is a single contiguous range scan.
- **type second** groups an object's records by kind: its inode record sorts before its extent records, which sort before its attribute records.
- **offset third** orders records within a kind, and its meaning depends on the type.

7.2 The polymorphic offset

The third field is interpreted according to type:

type	offset meaning
EMBK_TYPE_INODE	0 (there is one inode per object; nothing to disambiguate).
EMBK_TYPE_DIR_ENTRY	Hash of the entry name, so a name maps directly to a key.
EMBK_TYPE_EXTENT	Byte offset within the file, so extents sort in file order.
EMBK_TYPE_XATTR	Hash of the attribute name.

The same field means “file position” for an extent, “name hash” for a directory entry, and zero for an inode. One key structure, made polymorphic by type, addresses every kind of record. The upper seven bytes of type are reserved, leaving room to grow the type space or add sub-typing without changing the layout.

8 Tree Nodes

Metadata is stored in copy-on-write B-trees whose blocks are **nodes**. Every node, whether internal or leaf, begins with the same 40-byte header. The header's level field both distinguishes leaves from internal nodes and records the node's height above the leaves.

8.1 Node header

```
struct embk_node_header {
    uint64_t checksum;     // CRC32C (v1) over bytes [8 .. block_size-1]; stored first
    uint64_t magic;        // EMBKFS_NODE_MAGIC -> reads "EMBKNODE" in a hex dump
    uint64_t generation;   // generation that wrote this node (cross-checked vs the pointer)
    uint64_t block;        // this node's own block number (cross-checked vs the pointer)
    uint8_t  level;        // 0 = leaf; >0 = internal, value = height above leaves
    uint8_t  reserved[3];  // padding / future flags; must be 0
    uint32_t nritems;      // number of items currently in this node
};
// = 40 bytes (8-aligned). Body occupies block_size - 40 bytes (4056 at 4 KiB).


#define EMBKFS_NODE_MAGIC  0x45444F4E4B424D45ULL  // little-endian: "EMBKNODE"
```

The header provides four independent checks on every node read, layered beneath the Merkle chain from the parent pointer:

- checksum detects bit-rot and torn writes within the node, and allows standalone validation (during a scrub) without the parent in hand.
- magic confirms the block is actually an EMBKFS node and not data or garbage.
- generation is cross-checked against the pointer's generation, catching a stale pointer to a reused block.
- block is cross-checked against the pointer's target, catching a misplaced or wrong-block read.

> **Why the checksum is first.** Placing checksum at offset 0 lets it cover the entire remainder of the node, including the rest of the header. The computation is simply: checksum bytes 8 through block_size-1 and store the result in bytes 0 through 7.

8.2 Internal nodes

An internal node routes searches. It holds a sorted run of {key, block pointer} slots; each slot's key is the smallest key present in the subtree under its pointer. A search binary-searches the keys to choose which child to descend into.

```
struct embk_internal_slot {
    struct embk_key      key;   // 24 bytes - smallest key in the child subtree
    struct embk_block_ptr ptr;   // 32 bytes - fat pointer to that child node
```

```
};
// = 56 bytes per slot.
```

Fan-out. With 4056 body bytes and 56 bytes per slot, an internal node holds up to 72 children. This shallow, wide shape keeps the tree near-flat:

Tree height	Leaf capacity (× many items each)
Height 2	72 leaves
Height 3	5,184 leaves
Height 4	373,248 leaves
Height 5	~26.8 million leaves

Each leaf holds many items, so a filesystem of millions of objects is only four or five levels deep — any lookup from root to leaf touches just four or five blocks. This is why the unified-tree model scales to the volume sizes EMBKFS targets.

8.3 Leaf nodes

Leaves hold the actual records, which are different sizes (an inode is fixed at 128 bytes; a directory entry varies with the name length; an extent is 64 bytes). A leaf therefore cannot be an array of fixed slots. EMBKFS uses a **slotted-page** layout that grows from both ends of the block toward the middle:

- **From the front** (just after the header): a sorted array of fixed-size item headers, each carrying a key and the location and length of its data. These grow toward higher addresses as items are added.
- **From the back** (the end of the block): the variable-size item data itself, growing toward lower addresses.
- **The free space** is the shrinking gap in the middle. The leaf is full when a new header and its data would overlap that gap.

```
struct embk_item_header {
   struct embk_key key;     // 24 bytes - item key (front array stays sorted by this)
   uint32_t       offset;  // item data location, measured from the START of the block
   uint32_t       size;    // item data length in bytes
};
// = 32 bytes.


// Leaf invariants:
//   item headers occupy [40, 40 + nritems*32), sorted ascending by key
//   item data occupies the tail; each item's bytes are at [offset, offset+size)
//   free space = (lowest item-data offset) - (40 + nritems*32), computed not stored
```

Because the item headers are fixed-size and sorted, binary search still works even though the data they point to is variable-size and scattered. Variable-size items pack to exactly the space they need. Insertion shifts only the small fixed headers; the data does not have to

stay in order, because the headers point to it. One leaf can hold an inode, several directory entries, and several extents together, each sized precisely. The leaf is type-agnostic storage of key-to-bytes; the key's type tells the reader how to interpret the bytes.

9 Item Types

Records stored in leaves are **items**. Each item's kind is given by the type field of its key. Lower type numbers sort first within an object, so the inode — which says what an object is — always comes before the object's contents. Gaps are left between assigned values so related types can be inserted later in the correct sort position without renumbering.

Type	Value	Meaning
EMBK_TYPE_INODE	1	Object metadata (one per object, offset = 0).
EMBK_TYPE_DIR_ENTRY	16	A name-to-object mapping inside a directory.
EMBK_TYPE_EXTENT	32	A run of file data (offset = file byte position).
EMBK_TYPE_XATTR	48	An extended attribute (offset = hash of the attr name).

9.1 Inode item

The inode carries an object's properties. Its key is {object_id, EMBK_TYPE_INODE, 0}. Crucially, an EMBKFS inode holds **no data pointers**: the locations of a file's data live in separate extent items sharing the same object_id. The inode says “I am a 12 KiB file”; the extent items say where those bytes are.

```
struct embk_inode_item {
    uint64_t size;        // object size in bytes
    uint64_t blocks;      // blocks actually allocated (du / sparse detection)
    uint64_t links;       // hard-link count (object freed when this hits 0)
    uint32_t mode;        // POSIX st_mode: object type + rwx/setuid/setgid/sticky
    uint32_t uid;         // owner user id
    uint32_t gid;         // owner group id
    uint32_t flags;       // per-object flags (immutable/compressed/...); reserved v1
    uint64_t atime;       // last access       - nanoseconds since the Unix epoch
    uint64_t mtime;       // last modification  - nanoseconds since the Unix epoch
    uint64_t ctime;       // last status change - nanoseconds since the Unix epoch
    uint64_t btime;       // creation (birth)   - nanoseconds since the Unix epoch
    uint64_t generation;  // object generation (reuse detection / file handles)
    uint8_t  reserved[40]; // future per-object attributes (ACL ref, quota id, ...); must be 0
};
// = 128 bytes.
```

The mode field follows the POSIX st_mode layout (object type in the high bits, permission bits in the low nine, plus setuid/setgid/sticky), so a future VFS and C library map onto it directly. Timestamps are nanoseconds since the Unix epoch in a single 64-bit field each, giving sub-

second precision and lasting past the year 2500. The inode is padded to a clean 128 bytes, reserving room for future per-object attributes — an ACL reference, a project/quota id, a compression policy — addable without a format change.

> **Timestamps and the clock.** EmbLink does not yet have a wall clock, so these timestamp fields are populated with placeholder values until one exists — the same situation as FAT's epoch dates. The format reserves the fields regardless, so no change is needed when a clock arrives.

9.2 Directory entry item

A directory entry maps a name to an object. Its key is {dir_object_id, EMBK_TYPE_DIR_ENTRY, name_hash}, where name_hash is a CRC32C-based hash of the name placed in the key's offset field. Finding an entry by name is therefore a single tree lookup: hash the name, look up the key. Because two names can hash alike, the entry stores the full name; a lookup that lands on a hash match compares the stored name to confirm.

```
struct embk_dir_entry_item {
    uint64_t target_object_id;  // the object this name refers to
    uint8_t  target_type;       // file/dir/symlink (duplicated from target inode)
    uint8_t  name_len;          // name length in bytes, 1..255 (POSIX NAME_MAX)
    uint8_t  reserved[6];       // padding / future flags; must be 0
    // immediately followed by name_len bytes of UTF-8 (raw bytes, byte-compared,
    // case-sensitive; NOT null-terminated)
};
// fixed part = 16 bytes; total item size = 16 + name_len.
```

The entry duplicates the target's target_type so that directory listing can show whether each entry is a file or a directory without reading every target's inode — a significant speedup for a one-byte cost. Names are UTF-8, stored as raw bytes and compared byte-for-byte; EMBKFS is therefore case-sensitive and encoding-agnostic, following Unix convention. Names are capped at 255 bytes, matching POSIX NAME_MAX; this revises the original draft's aspirational 4096-byte limit, which neither fits the leaf model nor reflects any real-world need.

9.3 Extent item

An extent locates a contiguous run of a file's data. Its key is {object_id, EMBK_TYPE_EXTENT, file_byte_offset}. Unlike FAT's cluster chains, which describe one cluster at a time, an extent describes a whole run as {disk block, length}: a perfectly contiguous one-gigabyte file needs a single extent, not a quarter-million chain entries. Because extents sort by file offset, a file's extents are stored in file order, and random access into a large file is a single tree lookup rather than a long chain walk.

```
struct embk_extent_item {
```

```
    uint64_t disk_block;     // first disk block of the run
    uint64_t length;         // run length, in blocks
    uint64_t logical_size;   // actual data length in bytes (last block may be partial)
    uint64_t checksum;       // CRC32C (v1) over the extent's DATA - verified on every read
    uint64_t generation;     // generation that wrote this extent (CoW provenance)
    uint32_t flags;          // HOLE (sparse) / COMPRESSED / ENCRYPTED - reserved v1
    uint32_t reserved0;      // padding; must be 0
    uint8_t  reserved1[16];  // future extent attributes (compression metadata); must be 0
};
// = 64 bytes.
```

End-to-end data integrity. EMBKFS checksums **file data**, not only metadata. Each extent carries a checksum over the data it locates, verified on every read. This is the guarantee that distinguishes a modern, reliable filesystem from a traditional one: silent data corruption is **detected**, and once redundancy is added, repairable. The cost is one hardware-accelerated CRC32C per data block. The `flags` field reserves a HOLE bit for sparse files (a run of zeros that occupies no disk) alongside future compression and encryption bits.

10 How the Pieces Compose

The structures above assemble into a complete filesystem:

- **A file** is one inode item plus a set of extent items locating its data — all sharing the file's object_id and clustered together in the tree.
- **A directory** is one inode item (with mode marking it a directory) plus directory-entry items mapping names to objects.
- **Path resolution** walks one directory at a time: hash the path component, look up the entry, confirm the name, follow target_object_id to the next directory.
- **Reading a file** looks up its inode for the size, then its extents in file order, reading and verifying the data blocks.

All of this is copy-on-write: an update writes new blocks from the leaf up to a new root, then atomically installs them by replacing the superblock's root pointer and bumping the generation. A crash before that final replacement leaves the previous tree wholly intact, so the filesystem is crash-consistent by construction. Integrity is end-to-end — a Merkle tree of checksums over metadata, plus per-extent checksums over data. A snapshot is simply an old root pointer retained rather than discarded.

11 Roadmap Beyond the Format

This specification defines the on-disk container and its contents. The machinery that operates on them, and the implementation, are sequenced as follows. Read-only implementation comes first, deliberately, to validate the format cheaply before the harder write-side work is built on top of it.

Phase	Topic	Description
1	Read-only implementation	A host-side formatter writes a known-good image; the kernel mounts it, walks the tree, and reads files. Validates the format before any write code exists.
2	Allocator	Free-space management, made snapshot-aware (a block referenced by a snapshot cannot be reused). The foundation of all writing.
3	Copy-on-write commit path	Write new nodes bottom-up, install a new root atomically, bump the generation. Minimal object-id allocation falls out here.
4	Transactions & checkpoints	Batching many changes per commit; the dual-superblock commit; replay-on-mount for crash recovery.
5	Snapshots & full root tree	The multi-root “tree of trees” enabling many snapshots. Beyond version 1.

Deferred / out of scope for v1: compression (LZ4/ZSTD per extent), encryption (AES-256-XTS), deduplication, multi-device storage pools, and cryptographic-hash checksums. Each is gated behind a feature flag so the format can adopt it without breaking existing volumes — which is exactly what the three-class feature mechanism was designed for.

EMBKFS Specification v2.0 — Foundations & Core Structures. A living document, versioned alongside the EmbLink operating system.